

Práctica de Optimización de Alto Rendimiento

Objetivos de la práctica:

-  Comprender cómo diferentes transformaciones del código fuente influyen en el desempeño de un programa, analizando su impacto en el uso del procesador y de la jerarquía de memoria. En particular, se aborda la reducción del trabajo computacional, la mejora del acceso a datos, el incremento del paralelismo de nivel de instrucciones (ILP) y la explotación del paralelismo de datos mediante instrucciones vectoriales (SIMD).

Bibliografía:

-  Introduction to High Performance Computing for Scientists and Engineers. CRC Press. Hager, Wellein. 2011.
-  Temas de jerarquía de memoria:
-  Del libro de Grama: 2.2.1 y 2.2.2.
-  Lecturas alternativas en sección 2.1.2 del libro de Dongarra y 2.7 del libro de Rauber.
-  Software Optimization Guide for AMD Family 15h Processors. https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/software-optimization-guides/47414_15h_sw_opt_guide.pdf
-  Intel 64 and IA-32 Architectures Optimization Reference Manual <https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf>
-  Intel Intrinsics Guide: <https://software.intel.com/sites/landingpage/IntrinsicsGuide/>

Instrucciones para la resolución de los ejercicios:

1. ⚡ Para poder observar de forma aislada el efecto de las optimizaciones manuales realizadas en los ejercicios, compilar los programas sin utilizar las optimizaciones automáticas del compilador.
2. ⛔ No realizar optimizaciones adicionales (por ejemplo, simplificaciones matemáticas) a las indicadas en cada ejercicio.
3. ⌚ Para medir el tiempo de ejecución de los programas, anteponer el comando *time* al nombre de su programa ejecutable en el script. Observar el tiempo indicado como "real", que indica el tiempo real de ejecución tal como si se midiera con un cronómetro, dentro del archivo de error.
4. 💻 Preferentemente, utilizar el cluster para ejecutar los programas.

Ejercicios

1) Describir los niveles de optimización 0, 2 y 3 de GCC o ICC.

✓ [Ver solución →](#)

2) Técnica: "Hacer menos trabajo y más liviano".

✓ [Ver solución →](#)

1. 🛠 Solamente aplicando el sentido común, hacer que el siguiente código de programa realice menos trabajo. Dado que el beneficio de esta técnica es trivial, no se evaluará el tiempo de ejecución de este programa (además, por esta razón, el código está incompleto).

```
#include <stdio.h>
int flag = 0;
for (i = 0; i < 1000; i++) {
    if (complex_func(a[i]) < 55)
        flag = 1;
}

printf("¿Elemento encontrado? %d\n", flag);
```

1.  Utilizando operaciones más livianas (menos complejas), y sin modificar la estructura del algoritmo, optimizar el siguiente código de programa. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

```
int main() {
float i, j, a;
for (i = 0; i < 1000; i++)
    for (j = 0; j < 1000000; j++)
        a = i + j;    return 0;
}
```

3) Técnica: "Reducción del almacenamiento para datos".

 **Ver solución →**

El siguiente programa representa una grilla con células vivas y muertas del Juego de la Vida. El programa arma una estructura de datos que permite almacenar el estado de cada célula (cero para célula muerta y uno para célula viva).

1.  Realizar un diagrama que permita observar punteros y bloques de memoria de la estructura de datos utilizada para representar la grilla. ¿Qué ventajas tiene el uso de esa estructura frente a una declaración de una matriz estándar?
2.  Optimizar el programa mediante la técnica de reducción del almacenamiento para datos.
3.  Contrastar el programa optimizado frente a la versión original en relación al tiempo de ejecución y memoria utilizada para almacenar la matriz.

```
#include <stdlib.h>
#define X 20000
#define Y 20000
int main() {
    int i, j;
    int **m = malloc(X * sizeof(int*));
    int *x = malloc(X * Y * sizeof(int));
    for (i = 0; i < X; i++)
        m[i] = &x[i*Y];

    for(i = 0; i < X; i++)
        for(j = 0; j < Y; j++)
            m[i][j] = j % 2;

    return 0;
}
```

4) Técnica: "Código en Línea".

✓ [Ver solución →](#)

Optimizar el siguiente programa utilizando la técnica de Código en Línea (se requiere eliminar la función y escribir el código en donde se encuentra cada a la función). Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original. ¿En qué casos cree conveniente utilizar funciones y en qué casos no? analizar en función de: funciones grandes y pequeñas, posibilidad de optimizaciones globales, incremento del tamaño del código.

```
int pordos (int x) {
    return x * 2;
}

int main() {
    register unsigned int i, res = 0;
    for (i = 0; i < 500000000; i++)
        res += pordos(i);
    return 0;
}
```

5) Técnica: "Desenrollado de bucles".

✓ [Ver solución →](#)

El programa 1 es el original. El programa 2 resulta de aplicar la técnica de desenrollado de bucles. El programa 3, luego de aplicar desenrollado de bucles, aplicó una técnica para reducir dependencias entre instrucciones dentro de cada iteración. Ejecutar los programas y, tras observar los tiempos de ejecución de cada uno, formular hipótesis que expliquen las diferencias de tiempos.

Programa 1

```
#include <stdio.h>

int main() {
    register int i;
    register double a = 0;
    for (i = 0; i < 400000000; i++)
        a+= 0.00000001;
    printf("El valor calculado es: %f\n", a);
    return 0;
}
```

Programa 2

```
#include <stdio.h>

int main() {
    register int i;
    register double a;
    for (i = 0; i < 400000000; i+=4) {
        a+= 0.00000001;
        a+= 0.00000001;
        a+= 0.00000001;
        a+= 0.00000001;
    }
    printf("El valor calculado es: %f\n", a);
    return 0;
}
```

Programa 3

```
#include <stdio.h>

int main() {
    register int i;
    register double a = 0, a1 = 0, a2 = 0, a3 = 0;
    for (i = 0; i < 400000000; i+=4) {
        a+= 0.00000001;
        a1+= 0.00000001;
        a2+= 0.00000001;
        a3+= 0.00000001;
    }
    a+=a1+a2+a3;
    printf("El valor calculado es: %f\n", a);
    return 0;
}
```

6) Técnica: "Extracción de subexpresiones comunes".

✓ [Ver solución →](#)

Optimizar el siguiente programa extrayendo la subexpresión común. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

```
#include <stdio.h>

int main() {
    register int i, j;
    register int a = 0, a1 = 0, a2 = 0, a3 = 0;
    for (i = 0; i < 1000; i++)
        for (j = 0; j < 1000000; j+=4) {
            a += i * i * i + j;
            a1 += i * i * i + j + 1;
            a2 += i * i * i + j + 2;
            a3 += i * i * i + j + 3;
        }
    a += a1 + a2 + a3;
    printf("El valor calculado es: %d\n", a);
    return 0;
}
```

7) Técnica: "Evitar saltos condicionales".

✓ [Ver solución →](#)

Optimizar el siguiente programa mediante la evitación de saltos. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original. Formular hipótesis que expliquen las diferencias observadas en los tiempos de ejecución.

```
#include <stdlib.h>

#define X 1000000

int main() {
    int i, j, **m;
    m = malloc(900 * sizeof(int*));
    for (i = 0; i < 900; i++)
        m[i] = malloc(X * sizeof(int));
    for(i = 0; i < 900; i++)
        for(j = 0; j < X; j++)
            switch (i) {
                case 0 ... 299: m[i][j] = 0; break;
                case 300 ... 599: m[i][j] = 1; break;
                default: m[i][j] = 2; break;
            }
    return 0;
}
```

8) Técnica: "Uso eficiente de la caché: localidad espacial y temporal".

Ver solución →

1.  Optimizar el siguiente programa buscando aumentar la localidad espacial y temporal en el acceso a datos de una matriz. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.
2.  Para el procesador utilizado en el cluster, indicar los tamaños de:
3.  memoria caché de nivel 3. \$ cat /sys/devices/system/cpu/cpu0/cache/index2/size o \$ lscpu
4.  línea de caché de nivel 3. \$ cat /sys/devices/system/cpu/cpu0/cache/index2/coherency_line_size
5.  Calcular, de manera analítica, cuántos fallos de caché hay en cada una de las dos versiones del programa (original y optimizada). Para simplificar,

consideraremos solamente los fallos de caché de nivel 3, y un tamaño de 128 KiB para ese nivel.

```
#include <stdlib.h>

#define X 50000
#define Y 10000

int main() {
    int i, j;
    int **m;
    m = malloc(X * sizeof(int*));
    for (i = 0; i < X; i++)
        m[i] = malloc(Y * sizeof(int));
    for(j = 0; j < Y; j++)
        for(i = 0; i < X; i++)
            m[i][j] = i + j;
    return 0;
}
```

9) + Suma horizontal con instrucciones vectoriales.

✓ [Ver solución →](#)

Se requiere implementar un programa en C que realice la suma horizontal de un vector de floats (ejemplo, dado el vector {10.5, 5.5, 4}, el resultado de la suma horizontal es 20). El vector es muy grande (se requiere memoria dinámica) y el programa debería funcionar para diversos tamaños de vector, simplemente modificando el valor de un literal mediante la directiva `#define`.

Resolver los siguientes incisos:

1.  Implementar una solución utilizando instrucciones escalares.
2.  Implementar una solución utilizando instrucciones vectoriales, considerando juegos de instrucciones disponibles que utilicen registros de mayor tamaño, e instrucciones que trabajen con posiciones de memoria alineadas para obtener un mayor rendimiento.
3.  Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

Notas sobre la compilación del programa vectorial: Al compilar es necesario indicar el flag correspondiente al juego de instrucciones utilizado (por ejemplo, para avx: `-mavx`).

10) Operaciones vectoriales con funciones intrínsecas.

 [Ver solución →](#)

Dado el siguiente código de programa que utiliza operaciones escalares, escribir su equivalente con funciones intrínsecas vectoriales. Utilizar los juegos de instrucciones disponibles que utilicen registros de mayor tamaño.

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int i, elementos = 100000000;
    int *v;
    v = malloc(elementos * sizeof(int));
    //no vectorizar este for
    for (i = 0; i < elementos; i++)
        v[i] = i;
    for (i = 0; i < elementos; i++)
        if (v[i] > 500000000)
            v[i] = v[i] - 5;
        else
            v[i] = v[i] + 5;
    printf("elemento 0: %d\n", v[0]);
    printf("elemento 499999999: %d\n", v[499999999]);
    printf("elemento 500000000: %d\n", v[500000000]);
    printf("elemento 999999999: %d\n", v[999999999]);
    return 0;
}
```



Ejercicio Adicional

Operaciones vectoriales con condicionales

 [Ver solución →](#)

Dado el siguiente código de programa que utiliza operaciones escalares, escribir su equivalente con funciones intrínsecas vectoriales. Utilizar los juegos de instrucciones disponibles que utilicen registros de mayor tamaño.

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int i, elementos = 1000005;
    int *v;
    v = malloc(elementos * sizeof(int));
    // no vectorizar este for
    for (i = 0; i < elementos; i++)
        v[i] = rand() % 10000;
    printf("v[0]=%11d, ", v[0]);
    printf("v[11]=%11d, ", v[11]);
    printf("v[elementos-1]=%11d\n", v[elementos-1]);
    for (i = 0; i < elementos; i++) {
        if(2 * v[i] < i * 10)
            v[i] = v[i] * 2;
        else
            v[i] = (v[i] - 2) * -2;
    }
    printf("v[0]=%11d, ", v[0]);
    printf("v[11]=%11d, ", v[11]);
    printf("v[elementos-1]=%11d\n", v[elementos-1]);
    return 0;
}
```



Soluciones

Soluciones - Práctica de Optimización de Alto Rendimiento

Ejercicio 1: Niveles de Optimización de GCC

Descripción de los niveles `-00`, `-02` y `-03`

`-00` (por defecto)

- Reduce el tiempo de compilación
 - Depuración produce resultados esperados
 - Ninguna optimización activada
-

`-02` (recomendado para producción)

- Casi todas las optimizaciones que no implican trade-off tamaño/velocidad
- Mayor tiempo de compilación y mejor rendimiento
- Incluye todo `-01` + flags como:
 - `-falign-functions`
 - `-fcse-follow-jumps`
 - `-fgcse`
 - `-finline-functions`
 - `-fstrict-aliasing`
 - `-ftree-loop-vectorize`
 - `-ftree-pre`
 - `-ftree-vrp`

- Entre otros (~50 flags adicionales)

-03 (máxima optimización)

- Incluye todo `-02` +:

Flag	Descripción
<code>-fgcse-after-reload</code>	Optimización GCSE después del reload
<code>-fipa-cp-clone</code>	Clonado de propagación de constantes IPA
<code>-floop-interchange</code>	Intercambio de bucles
<code>-floop-unroll-and-jam</code>	Desenrollado y fusión de bucles
<code>-fpeel-loops</code>	Pelado de bucles
<code>-fpredictive-commoning</code>	Commoning predictivo
<code>-fsplit-loops</code>	División de bucles
<code>-fsplit-paths</code>	División de caminos
<code>-ftree-loop-distribution</code>	Distribución de bucles
<code>-ftree-partial-pre</code>	PRE parcial en árboles
<code>-funswitch-loops</code>	Descondicionalización de bucles
<code>-fvect-cost-model=dynamic</code>	Modelo de costo de vectorización dinámico
<code>-fversion-loops-for-strides</code>	Versionado de bucles para strides

Ejercicio 2: Técnica "Hacer menos trabajo y más liviano"

2.1) Reducción de trabajo

Archivo: 2Aex.c

Solución: Se optimiza el bucle para que termine temprano cuando encuentra un elemento que cumple la condición, en lugar de recorrer todo el array innecesariamente.

```
int solucionOptima(){
    int flag = 0;
    int aMax = n;
    while(!flag && i < aMax){
        flag = complex_func(a[i]) < 55;
        i++;
    }
    printf("¿Elemento encontrado? %d\n", flag);
    return 0;
}
```

Optimización aplicada: Early exit (salida temprana) - una vez que se encuentra un elemento que satisface la condición (`complex_func(a[i]) < 55`), el bucle termina, evitando iteraciones innecesarias.

2.2) Operaciones más livianas

Archivo: 2Bex.c

Solución: Se cambia el tipo de dato de `float` a `int` para las variables de iteración y resultado, ya que las operaciones con enteros son más livianas que con punto flotante.

Solución Inicial: 5,524s

Solución Nueva: 3,476s

```
int solucionNueva(){
    int i, j, a;

    for(i = 0; i < 1000; i++){
        for(j = 0; j < 1000000; j++){
            a = i + j;
        }
    }
    return a;
}
```

Optimización aplicada: Reemplazo de operaciones de punto flotante por operaciones con enteros, que son computacionalmente menos costosas.

Ejercicio 3: Técnica "Reducción del almacenamiento para datos"

3.1) Diagrama de la estructura de datos

A continuación se muestra el diagrama que representa punteros y bloques de memoria de la estructura de datos utilizada para la grilla:

Diagrama de la estructura de datos

Ventajas de usar un solo bloque de memoria:

- **Datos alineados:**
 - La CPU accede de forma rápida y eficiente, mejorando el rendimiento.
 - Menos saltos de memoria, aprovechando de mejor forma la caché y reduciendo el tiempo de ejecución.
 - **Memoria dinámica:** permite variar el tamaño de la memoria durante la ejecución.
-

3.2) Optimización mediante reducción del almacenamiento

Descripción: La reducción de datos consiste en achicar el size del tipo de dato ya que solo guardaremos estados booleanos. El tipo de dato mínimo es `char` que ocupa 1 byte, en vez de `int` que ocupa 4 bytes.

Archivo original (3-2ex.c): Utiliza `int` para almacenar valores booleanos (0 o 1).

Archivo optimizado (3-20ptex.c): Utiliza `char` para almacenar los mismos valores, reduciendo el consumo de memoria.

3.3) Comparación de rendimiento y memoria

Tiempo de ejecución

Versión	Real	User	Sys
Con <code>int</code>	0m2,648s	0m2,034s	0m0,612s
Con <code>char</code>	0m2,144s	0m1,984s	0m0,156s

Mejora: ~0,5s en tiempo real.

Uso de memoria

Tipo de dato	Cálculo	Bytes	MiB
<code>int</code> (4 bytes)	$(20000^2) \times 4$	1.600.000.000	~1526 MiB
<code>char</code> (1 byte)	$(20000^2) \times 1$	400.000.000	~381 MiB

Ahorro de memoria: ~1145 MiB (75% menos memoria).

Ejercicio 4: Técnica "Código en Línea"

Descripción

Se optimiza el siguiente programa eliminando la función y escribiendo el código directamente donde se realiza la llamada:

```
// Versión original con llamada a función
int pordos(int x){
    return x*2;
}

// Versión optimizada con código en línea
res += (i*2); // en lugar de res += pordos(i);
```

Resultados de ejecución (sin optimización del compilador)

Versión	Tiempo real
Con llamada a función	0m1,765s
Con código en línea	0m0,255s

Diferencia: 1,51s a favor del código en línea.

Resultados con optimización -03

Versión	Tiempo real
Con llamada a función	0m0,006s
Con código en línea	0m0,004s

Diferencia: Mínima (0,002s).

Conclusión

- **Llamada por funciones:** Tiene cierto overhead (peso) que demora la ejecución al realizar el llamado.
- **Código en línea:** Al no tener llamadas de función, elimina los tiempos de demora asociados al salto y retorno.
- **Recomendación:** Es preferible usar funciones cuando se trata de procedimientos más complejos, donde el overhead es despreciable frente al trabajo realizado. Para funciones pequeñas y frecuentemente llamadas, el código en línea ofrece mejor rendimiento.
- **Con optimización -O3 :** El compilador realiza inline automático, por lo que la diferencia se vuelve mínima.

Ejercicio 5: Técnica "Desenrollado de bucles"

Programas analizados

Programa 1 (Original)

```
register int i;
register double a = 0;
for (i = 0; i < 400000000; i++)
    a+= 0.00000001;
```

Programa 2 (Desenrollado de bucles)

```
register int i;
register double a = 0;
for (i = 0; i < 400000000; i+=4) {
    a+= 0.00000001;
    a+= 0.00000001;
    a+= 0.00000001;
    a+= 0.00000001;
}
```

Programa 3 (Desenrollado + reducción de dependencias)

```
register int i;
register double a = 0, a1 = 0, a2 = 0, a3 = 0;
for (i = 0; i < 400000000; i+=4) {
    a+= 0.00000001;
    a1+= 0.00000001;
    a2+= 0.00000001;
    a3+= 0.00000001;
}
a+=a1+a2+a3;
```

Resultados de ejecución

Programa	Tiempo real	Observaciones
Programa 1 (Original)	0m0,148s	Operaciones dependientes, no permite paralelización a nivel de instrucciones
Programa 2 (Desenrollado)	0m0,132s	Reduce los saltos de bucle 4 veces
Programa 3 (Desenrollado + variables independientes)	0m0,035s	Permite paralelizar instrucciones al eliminar dependencias

Análisis

1. **Programa 1:** Es el más lento debido a que las operaciones son dependientes entre sí, lo que impide la paralelización a nivel de instrucciones (ILP).
2. **Programa 2:** Mejora ligeramente el tiempo al reducir la cantidad de saltos de bucle (se itera 4 veces menos). Sin embargo, mantiene la dependencia en la variable `a`.
3. **Programa 3:** Es el más rápido porque:
4. Logra el desenrollado de bucles (menos saltos).
5. Usa variables independientes (`a`, `a1`, `a2`, `a3`) para prevenir la dependencia de datos.

6. Permite al procesador paralelizar las instrucciones, aprovechando el ILP (Instruction Level Parallelism).

Ejercicio 6: Técnica "Extracción de subexpresiones comunes"

Descripción

Se optimiza el siguiente programa extrayendo la subexpresión común `i * i * i` que se recalcula en cada iteración del bucle interno.

Código original (`6ex.c`)

```
for (i = 0; i < 1000; i++)
    for (j = 0; j < 1000000; j+=4) {
        a += i * i * i + j;
        a1 += i * i * i + j + 1;
        a2 += i * i * i + j + 2;
        a3 += i * i * i + j + 3;
    }
```

Código optimizado (`6exOptimized.c`)

```
for (i = 0; i < 1000; i++){
    int c = i * i * i;           // extraído fuera del bucle interno
    for (j = 0; j < 1000000; j+=4) {
        int d = c + j;          // subexpresión común calculada una vez
        a  += d;
        a1 += d + 1;
        a2 += d + 2;
        a3 += d + 3;
    }
}
```

Optimización aplicada

- `i * i * i` se calcula una sola vez por iteración externa y se almacena en `c`.
- `c + j` se calcula una sola vez por iteración interna y se almacena en `d`.
- Se evitan **4 multiplicaciones triples** y **4 sumas redundantes** por cada iteración del bucle interno.

Resultados de ejecución

Versión	Resultado	Tiempo real
Original	20242176	0m1,499s
Optimizada	20242176	0m1,049s

Mejora: ~0,45s (~30% más rápido).

Conclusión

La extracción de subexpresiones comunes, si bien genera cierta dependencia (las variables `a` esperan a `d`, y `d` espera a `c`), reduce el tiempo total en aproximadamente medio segundo, ya que evita que cada instrucción recalculé de nuevo el mismo valor.

Ejercicio 7: Técnica "Evitar saltos condicionales"

Programas analizados

Código original (`7ex.c`)

```
for(i = 0; i < 900; i++)
    for(j = 0; j < X; j++)
        switch (i) {
            case 0 ... 299:    m[i][j] = 0; break;
            case 300 ... 599: m[i][j] = 1; break;
            default:          m[i][j] = 2; break;
        }
```

Código optimizado (`7exOpt.c`)

```
for(i = 0; i < 900; i++){
    cond = (i >= 300) + (i >= 600);
    for(j = 0; j < X; j++)
        m[i][j] = cond;
}
```

Optimización aplicada

Se reemplaza el `switch` con rangos por una **expresión aritmética sin saltos condicionales**: - `(i >= 300)` evalúa a 1 si `i >= 300`, sino 0 - `(i >= 600)` evalúa a 1 si `i >= 600`, sino 0 - La suma de ambas produce directamente el valor correcto: 0, 1 o 2

Esto elimina los **branch mispredictions** que genera el `switch`, permitiendo que el pipeline del procesador fluya sin interrupciones.

Resultados de ejecución

Versión	Real	User	Sys
Original	0m7,291s	0m5,152s	0m2,136s
Optimizada	0m6,385s	0m4,396s	0m1,988s

Mejora: ~0,9s (~12% más rápido).

Conclusión

El `switch` con rangos genera saltos condicionales que el procesador debe predecir. Cuando la predicción falla (branch misprediction), el pipeline se descarga y se pierde rendimiento. Al reemplazarlo por una expresión aritmética pura, se elimina por completo esta fuente de overhead, logrando una ejecución más rápida y predecible.

Ejercicio 8: Técnica "Uso eficiente de la caché: localidad espacial y temporal"

8.1) Optimización del programa para mejorar localidad

Archivo: 8P/8ex.c

Código original (`solucionInicial`)

```
for(j = 0; j < Y; j++)
    for(i = 0; i < X; i++)
        m[i][j] = i + j;
```

Código optimizado (`solucionOptimizada`)

```
for(j = 0; j < Y; j++)
    for(i = 0; i < X; i++)
        m[j][i] = i + j;
```

Optimización aplicada: Se corrigió el patrón de acceso a la matriz. En la versión original, los bucles recorren la matriz por columnas (`m[i][j]`), lo que genera saltos de memoria aleatorios ya que C almacena arrays en orden row-major. En la versión optimizada, se accede secuencialmente (`m[j][i]`), aprovechando la **localidad espacial** y temporal de la caché.

Resultados de ejecución

Versión	Real	User	Sys
Original (acceso por columnas)	0m45,895s	0m44,004s	0m1,876s
Optimizada (acceso por filas)	0m3,251s	0m2,803s	0m0,444s

Mejora: ~42,6s (~93% más rápido, factor de ~14x).

8.2) Características del procesador del cluster

Parámetro	Valor	Comando
Memoria caché L3	256K	<code>cat /sys/devices/system/cpu/cpu0/cache/index2/size</code>
Línea de caché L3	64 bytes	<code>cat /sys/devices/system/cpu/cpu0/cache/index2/coherency_line_size</code>

8.3) Cálculo analítico de fallos de caché

Datos: - Matriz de $X \times Y = 50000 \times 10000 = 5 \times 10^8$ elementos - Cada entero ocupa 4 bytes - Línea de caché = 64 bytes $\rightarrow 64/4 = 16$ enteros por línea

Versión original (acceso por columnas)

Al recorrer por columnas ($m[i][j]$ con i como índice interno), cada acceso salta a una fila diferente, provocando un **cache miss** en cada acceso:

- **Fallos de caché:** 5×10^8 (uno por cada elemento)

Versión optimizada (acceso por filas)

Al recorrer por filas ($m[j][i]$ con i como índice interno), se accede secuencialmente:

- Elementos por fila: 10000
- Fallos por fila: $10000 / 16 = 625$
- **Fallos totales:** $50000 \times 625 = 3.125 \times 10^7$

Comparación

Versión	Fallos de caché L3
Original	5×10^8
Optimizada	3.125×10^7

Reducción de fallos: 93.75% menos cache misses.

Conclusión

El acceso secuencial a memoria (row-major en C) aprovecha la localidad espacial: al cargar una línea de caché, se traen 16 elementos contiguos que se utilizan en las siguientes iteraciones. El acceso por columnas invalida constantemente la caché, mientras que el acceso por filas maximiza los hits, resultando en una mejora de rendimiento de ~14x.

Ejercicio 9: Suma horizontal con instrucciones vectoriales

9.1) Solución escalar (`9ex.c`)

Se implementa la suma horizontal de un vector de `N = 100.000.000` floats utilizando instrucciones escalares. Se utilizan 4 acumuladores independientes (`r1`, `r2`, `r3`, `r4`) para reducir dependencias entre instrucciones y permitir ILP:

```
#define N 100000000
#define Q 4

float *vector = malloc(N * sizeof(float));
// ... inicialización ...

float r1 = 0, r2 = 0, r3 = 0, r4 = 0;
int fin = N - (N % 4);

for (i = 0; i < fin; i += 4) {
    r1 += vector[i];
    r2 += vector[i + 1];
    r3 += vector[i + 2];
    r4 += vector[i + 3];
}

float r = r1 + r2 + r3 + r4;

// Elementos sobrantes
for (i = fin; i < N; i++) {
    r += vector[i];
}
```

Optimizaciones aplicadas: - **4 acumuladores independientes** para reducir dependencias de datos (ILP). - **Loop unrolling** de factor 4 para reducir overhead del bucle. - **Manejo de elementos sobrantes** para vectores cuyo tamaño no es múltiplo de 4.

9.2) Solución vectorial con SSE (9opt.c)

Se implementa la misma suma utilizando **instrucciones intrínsecas SSE** (registros de 128 bits = 4 floats por operación). Se utiliza **alineación de memoria** (`aligned_alloc(16, ...)`) para optimizar el acceso a caché.

```
#include <xmmintrin.h> // SSE -msse4.2

#define N 100000000

// Alineación a 16 bytes para SSE
float *vector = (float *)aligned_alloc(16, N * sizeof(float));

__m128 sum_vec = _mm_setzero_ps(); // Registro SSE inicializado a 0

int q = 4;
int fin = N - (N % q);

for (i = 0; i < fin; i += q) {
    __m128 data = _mm_load_ps(&vector[i]); // Cargar 4 floats alineados
    sum_vec = _mm_add_ps(sum_vec, data); // Sumar en paralelo
}

// Reducir los 4 valores del registro a un escalar
float sum_arr[q];
_mm_store_ps(sum_arr, sum_vec);
float suma_total = sum_arr[0] + sum_arr[1] + sum_arr[2] + sum_arr[3];

// Elementos sobrantes
for (i = fin; i < N; i++) {
    suma_total += vector[i];
}
```

Optimizaciones aplicadas: - **SSE 4.2** con registros de 128 bits → 4 floats procesados en paralelo. - `aligned_alloc(16, ...)` para alineación a 16 bytes, requerida por `_mm_load_ps`. - **Reducción horizontal** al final: se extraen los 4 acumuladores del registro SSE y se suman como escalar. - **Manejo de elementos sobrantes** para tamaños no múltiplos de 4.

Comando de compilación:

```
gcc -Wall -g -msse4.2 9opt.c -o 9opt
```

9.3) Comparación de rendimiento

Versión	Real	User	Sys
Escalar (9ex.c , -O0)	0m0,790s	0m0,637s	0m0,148s
Escalar (9ex.c , -O3)	(ver nota)		
Vectorial (9opt.c , -O3 -msse4.2)	0m0,303s	0m0,141s	0m0,157s

Mejora: ~0,487s (~62% más rápido, factor de ~2.6x).

Resultado verificado: ambas versiones producen 67108864.000000 (suma de 100.000.000 elementos con valor 1).

Análisis

La versión vectorial aprovecha instrucciones SIMD (Single Instruction, Multiple Data) para procesar 4 floats simultáneamente en un registro de 128 bits. Esto reduce la cantidad de instrucciones de carga y suma en un factor de 4. La alineación de memoria garantiza que `_mm_load_ps` acceda eficientemente a la caché L3, minimizando fallos. Además, la versión escalar también utiliza 4 acumuladores independientes, lo que permite ILP; por eso la mejora no es exactamente 4x, sino ~2.6x.

Ejercicio 10: Operaciones vectoriales con funciones intrínsecas

10.1) Solución escalar (10ex.c)

Se recorre un vector de 100.000.000 enteros aplicando una operación condicional:

```
for (i = 0; i < elementos; i++)
    if (v[i] > 500000000)
        v[i] = v[i] - 5;
    else
        v[i] = v[i] + 5;
```

10.2) Solución vectorial con SSE (`10ex0pt.c`)

Se implementa la misma operación utilizando **instrucciones intrínsecas SSE** (registros de 128 bits = 4 ints por operación). Se utiliza `posix_memalign` para alineación de memoria a 16 bytes.

La clave de esta solución es la **branchless vectorization**: en lugar de un `if/else` (que no se puede vectorizar directamente), se utiliza una **máscara de comparación** combinada con operaciones bit a bit:

```

#include <xmmintrin.h>

#define elementos 100000000

int *v;
posix_memalign((void**)&v, 16, elementos * sizeof(int));

// Constantes vectoriales
__m128i
    umbral = _mm_set1_epi32(500000000),
    suma5 = _mm_set1_epi32(5),
    resta5 = _mm_set1_epi32(-5);

for (i = 0; i < fin; i += 4) {
    __m128i
        // Cargar 4 enteros alineados
        datos = _mm_load_si128((__m128i*)&v[i]),

        // Comparación: 0xFFFFFFFF si datos[j] > umbral, 0x00000000 si no
        mascara = _mm_cmpgt_epi32(datos, umbral),

        // Seleccionar +5 o -5 según la máscara (branchless)
        ajuste = _mm_or_si128(
            _mm_and_si128(mascara, resta5),
            _mm_andnot_si128(mascara, suma5));

        // Aplicar el ajuste
        datos = _mm_add_epi32(datos, ajuste);

        // Guardar resultado
        _mm_store_si128((__m128i*)&v[i], datos);
}

// Elementos sobrantes
for (; i < elementos; i++) {
    v[i] = (v[i] > 500000000) ? v[i] - 5 : v[i] + 5;
}

```

Técnica de branchless con máscara: 1. `_mm_cmpgt_epi32(datos, umbral)` → produce `0xFFFFFFFF` donde la condición es verdadera, `0x00000000` donde es falsa. 2. `_mm_and_si128(mascara, resta5)` → selecciona `-5` donde la condición es verdadera. 3. `_mm_andnot_si128(mascara, suma5)` → selecciona `+5` donde la condición es falsa. 4. `_mm_or_si128(...)` → combina ambos resultados en un vector de ajustes. 5. `_mm_add_epi32(datos, ajuste)` → aplica el ajuste correcto a cada elemento.

Esto elimina completamente las ramas condicionales dentro del bucle vectorizado.

Comando de compilación:

```
gcc -Wall -g -msse4.2 10ex0pt.c -o 10opt
```

10.3) Comparación de rendimiento

Versión	Real	User	Sys
Escalar (10ex.c)	0m1,558s	0m1,410s	0m0,144s
Vectorial (10ex0pt.c , -msse4.2)	0m0,348s	0m0,187s	0m0,155s

Mejora: ~1,210s (~78% más rápido, factor de ~4.5x).

Resultados verificados (ambas versiones producen idéntica salida):

```
elemento 0: 5
elemento 499999999: 500000004
elemento 500000000: 500000005
elemento 999999999: 999999994
```

Análisis

La mejora de ~4.5x supera el factor teórico de 4x (4 elementos por registro SSE) gracias a la **eliminación de ramas condicionales**. En la versión escalar, el `if/else` genera branch mispredictions que descargan el pipeline. En la versión vectorial, la técnica de máscara + operaciones bitwise selecciona el ajuste correcto sin ninguna rama, combinando el beneficio del SIMD con el beneficio de evitar saltos condicionales (similar al Ejercicio 7).

Ejercicio Adicional: Operaciones vectoriales con condicionales

11.1) Descripción del problema

Se requiere optimizar un programa que aplica una operación condicional sobre un vector de enteros generados aleatoriamente. El programa original utiliza un `if/else` que no puede vectorizarse directamente.

Código original (`extra/extra.c`):

```
for (i = 0; i < elementos; i++) {
    if(2 * v[i] < i * 10)
        v[i] = v[i] * 2;
    else
        v[i] = (v[i] - 2) * -2;
}
```

11.2) Solución vectorial con SSE (`extra/extraVector.c`)

Se implementa la misma operación utilizando **instrucciones intrínsecas SSE** (registros de 128 bits = 4 ints por operación). Se utiliza `posix_memalign` para alineación de memoria a 16 bytes (128 bits).

La técnica clave es la **branchless vectorization**: se utiliza una **máscara de comparación** combinada con operaciones bit a bit para seleccionar entre las dos ramas sin utilizar saltos condicionales:

```

#include <xmmintrin.h>
#include <emmintrin.h>
#include <smmmintrin.h>

#define E      1000005
#define Size   (sizeof(int))
#define Q      (128/Size) // 4 elementos por registro SSE

int *v;
posix_memalign((void**)&v, Q*Size, E*Size); // Alineación a 16 bytes

// Constantes vectoriales
__m128i
    dosn    = _mm_set1_epi32(-2), // -2 para el cálculo alternativo
    diez    = _mm_set1_epi32(10); // 10 para la comparación

for (i = 0; i < fin; i += 4) {
    __m128i
        // Cargar 4 enteros alineados
        datos = _mm_load_si128((__m128i *)&v[i]),

        // 2 * v[i] (shift left 1 = multiplicación por 2)
        doble = _mm_slli_epi32(datos, 1),

        // i * 10 (vector de índices * 10)
        indices = _mm_set_epi32(i+3, i+2, i+1, i),
        multi    = _mm_mullo_epi32(indices, diez),

        // (v[i] - 2) * (-2)
        do2aux = _mm_add_epi32(datos, dosn),
        do2 = _mm_mullo_epi32(do2aux, dosn),

        // Comparación: 0xFFFFFFFF si 2*v[i] < i*10, 0x00000000 si no
        mascara = _mm_cmplt_epi32(doble, multi),

        // Seleccionar doble o do2 según la máscara (branchless)
        ajuste = _mm_or_si128(
            _mm_and_si128(mascara, doble),
            _mm_andnot_si128(mascara, do2)
        );

        // Guardar resultado
        _mm_store_si128((__m128i*)&v[i], ajuste);
}

// Elementos sobrantes
for (; i < E; i++) {
    if(2 * v[i] < i * 10)
        v[i] = v[i] * 2;
}

```

```

    else
        v[i] = (v[i] - 2) * -2;
    }

```

Técnica de branchless con máscara: 1. `_mm_cmplt_epi32(doble, multi)` → produce `0xFFFFFFFF` donde $2*v[i] < i*10$, `0x00000000` donde no. 2. `_mm_and_si128(mascara, doble)` → selecciona $2*v[i]$ donde la condición es verdadera. 3. `_mm_andnot_si128(mascara, do2)` → selecciona $(v[i]-2)*(-2)$ donde la condición es falsa. 4. `_mm_or_si128(...)` → combina ambos resultados en un vector. 5. Se almacena directamente el resultado en el vector.

Optimizaciones aplicadas: - **SSE 4.2** con registros de 128 bits → 4 enteros procesados en paralelo. - `posix_memalign` para alineación a 16 bytes, requerida por `_mm_load_si128`. - **Shift left** (`_mm_slli_epi32`) para multiplicación por 2 (más rápido que `_mm_mullo_epi32`). - **Branchless logic:** se eliminan las ramas condicionales mediante máscara de comparación. - **Manejo de elementos sobrantes** para tamaños no múltiplos de 4.

Comando de compilación:

```
gcc -Wall -g -msse4.2 extra/extraVector.c -o extraVector
```

11.3) Comparación de rendimiento

Versión	Real	User	Sys
Escalar (extra.c)	0m0,034s	0m0,030s	0m0,000s
Vectorial (extraVector.c)	0m0,027s	0m0,019s	0m0,005s

Mejora: `~0,007s` (~21% más rápido, factor de ~1.26x).

Resultados verificados (ambas versiones producen idéntica salida):

```

v[0]=      9383, v[11]=      27, v[elementos-1]=      9381
v[0]=    -18762, v[11]=     54, v[elementos-1]=    18762

```

Análisis

La mejora de $\sim 1.26x$ es menor que el factor teórico de $4x$ (4 elementos por registro SSE) debido a varios factores:

1. **Overhead de creación de índices:** En cada iteración se debe crear el vector de índices (`_mm_set_epi32(i+3, i+2, i+1, i)`) y multiplicarlo por 10, lo cual agrega instrucciones adicionales que no existen en la versión escalar (donde `i * 10` es una operación simple).
2. **Cálculo de ambas ramas:** Se calculan tanto `2*v[i]` como `(v[i]-2)*(-2)` en cada iteración, independientemente de la condición. Esto significa que se realiza más trabajo total que en la versión escalar, que solo evalúa una rama por iteración.
3. **Tamaño reducido del vector:** Con solo $\sim 1M$ de elementos, el vector entra completamente en la caché L3 del procesador, minimizando el impacto del acceso a memoria y haciendo que el overhead computacional sea más significativo en proporción.

A pesar de estos factores, la versión vectorial logra una mejora del 21% gracias a: - **Procesamiento SIMD:** 4 elementos procesados simultáneamente. - **Eliminación de ramas:** La técnica branchless evita branch mispredictions. - **Shift left:** La multiplicación por 2 mediante shift es más eficiente que la multiplicación convencional.